



Operating Systems

[1500WETOPS]

José Oramas



Processes and Threads

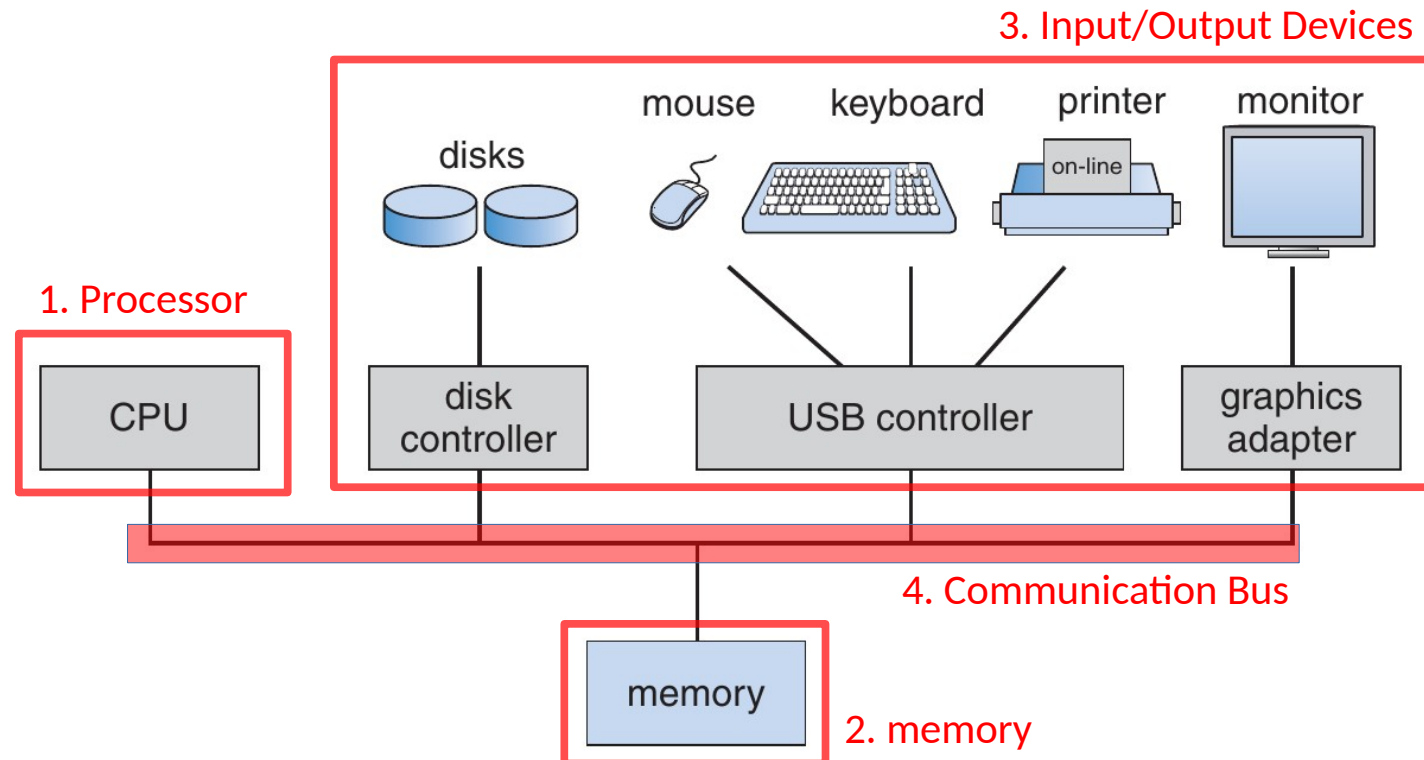
[Things in memory that are doing something]

José Oramas

Operating systems

A Modern Computer System

- Main components



Processor

Task

- Execute machine instructions
- Instructions: sets of bits of fixed length (e.g 16/32/64 bits)

Opcode

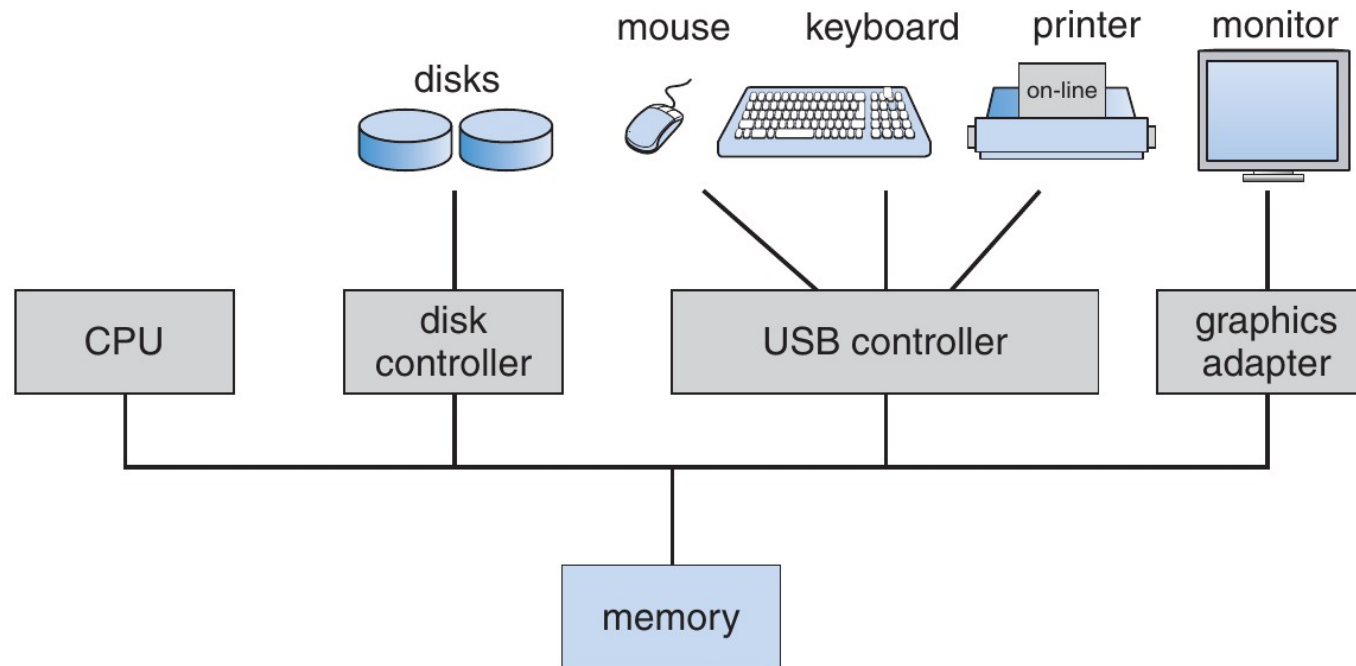
Memory Address

- Opcode: ADD, MOV, AND, POP, etc.
- Number of instructions:
 - **RISC**: reduced instruction set computer (50-tal)
 - **CISC**: complex instruction set computer (>300-400)

Processor

Interrupt Mechanism

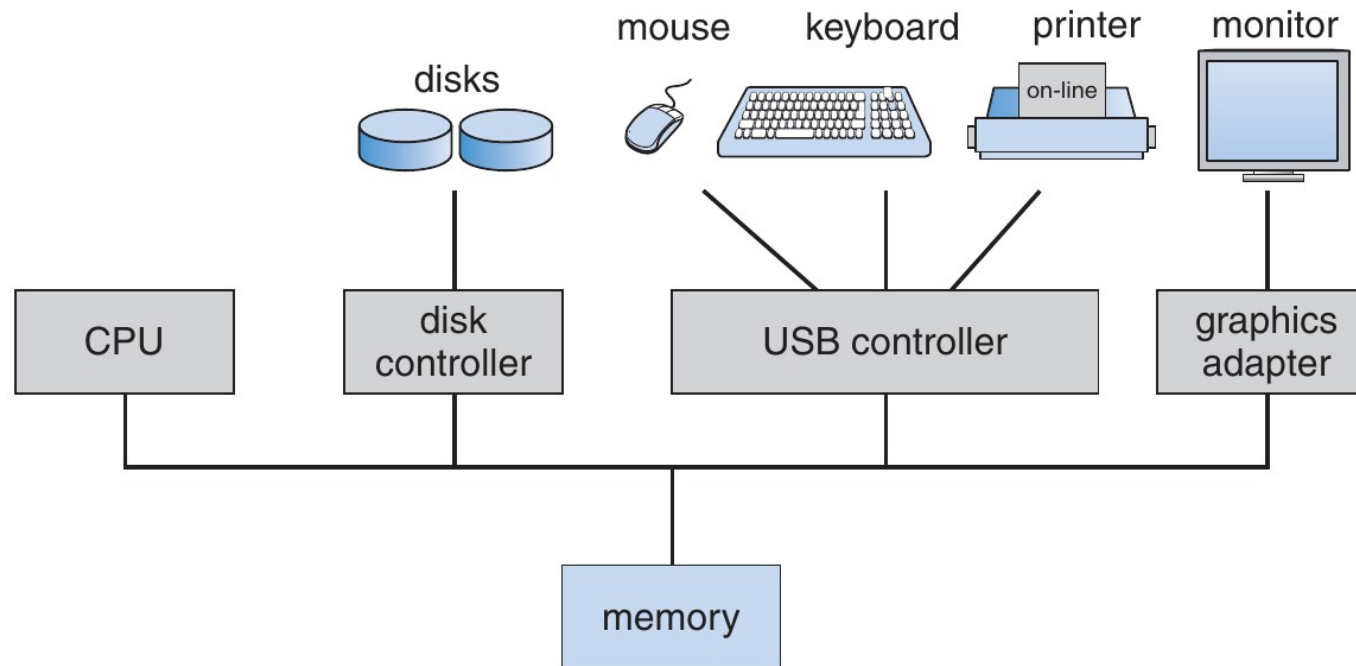
- The occurrence of an **event** is signaled through an **interrupt**



Processor

Interrupt Mechanism

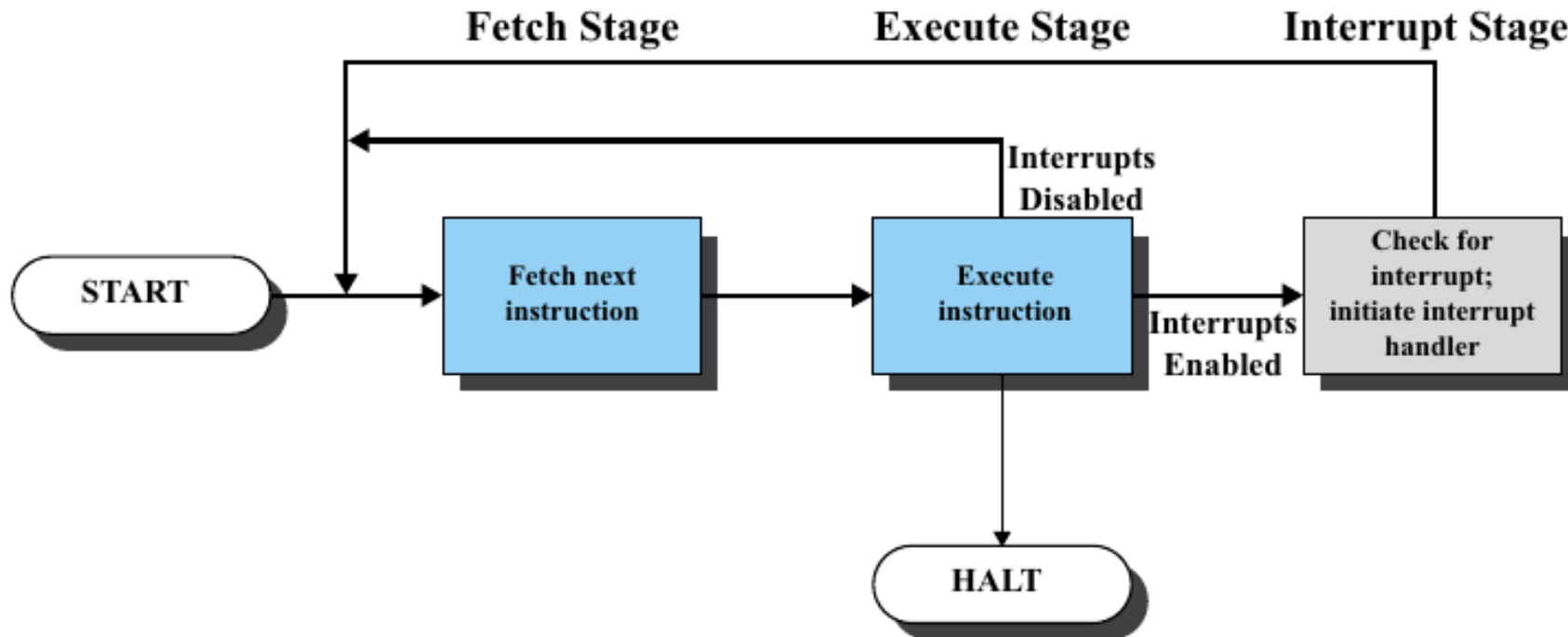
- The occurrence of an **event** is signaled through an **interruption**
- Done in software via a **system call** (monitor call)



Processor

Instruction Cycle

- Essential to maximize processor efficiency
→ Multi-program systems



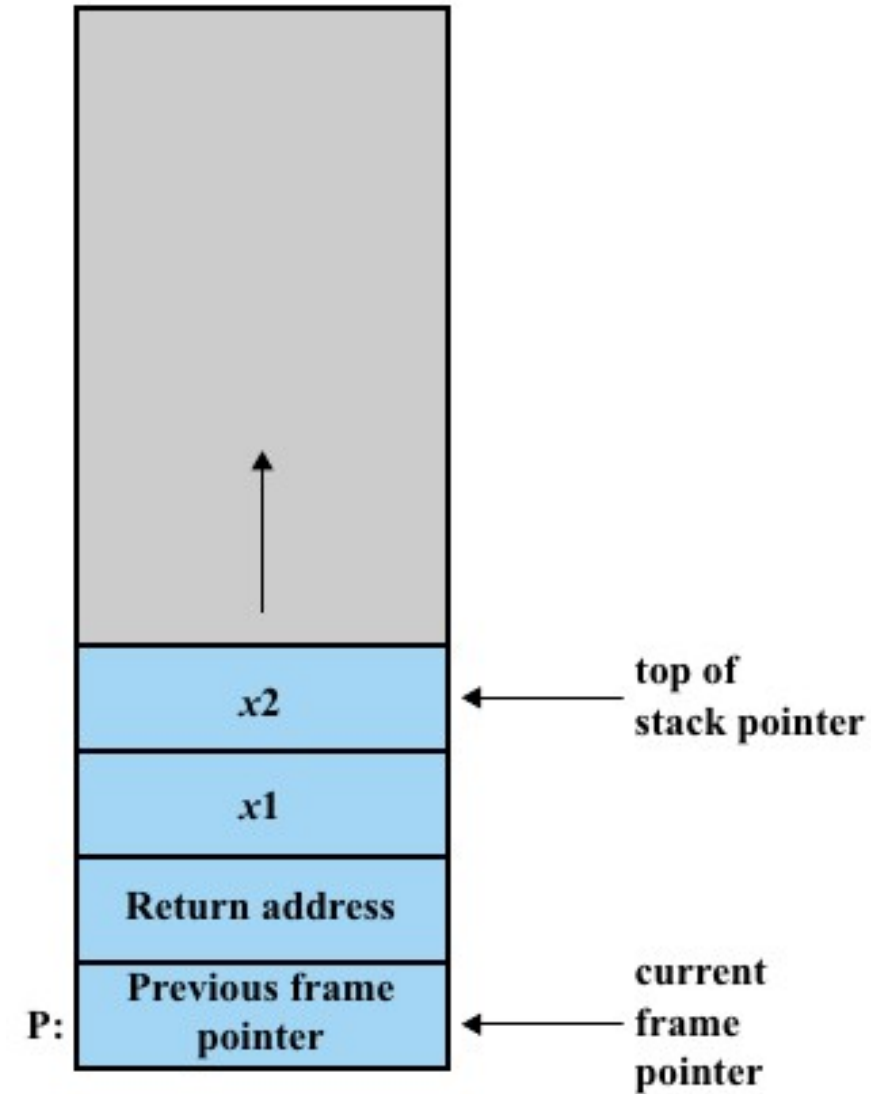
Processes

What?

- ~ “A program in progress”
 - Attributes
 - Text (code) of the process
 - Static/dynamic data to be processed
 - User stack (+kernel stack)
 - Process control block
- Together define the **process image**

Processes

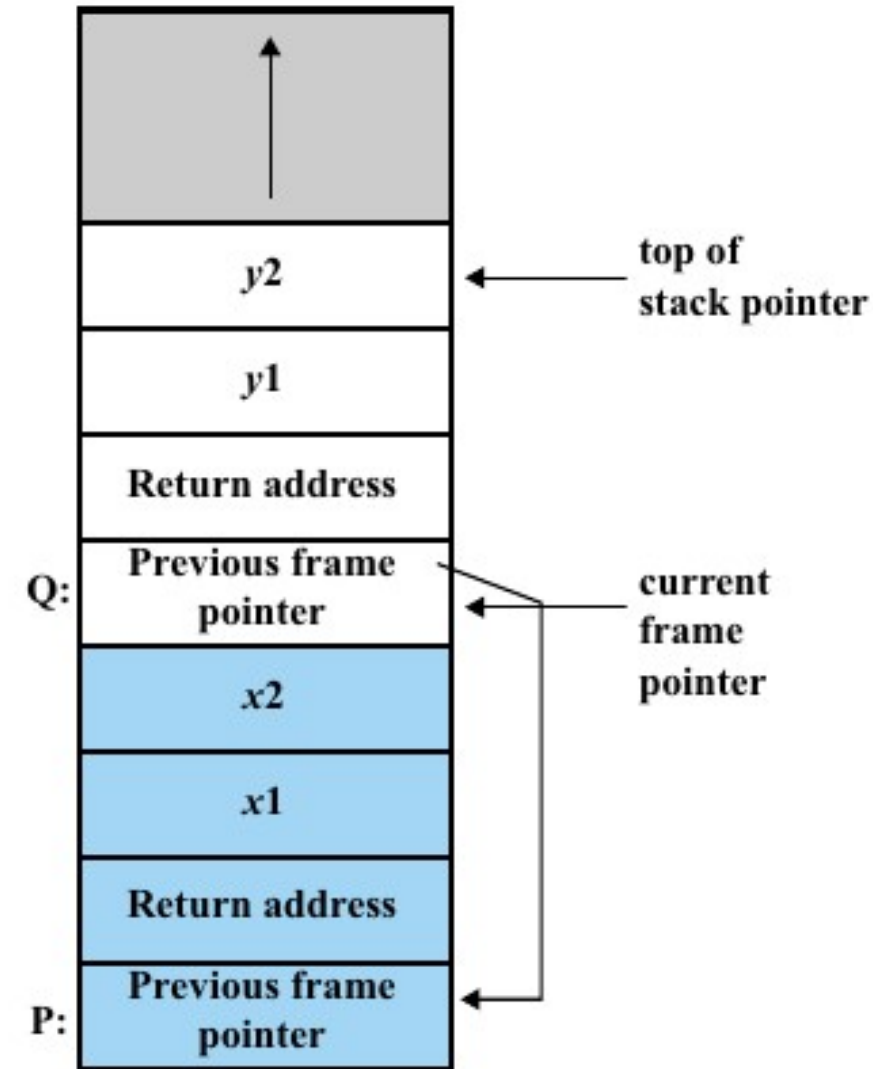
User Stack



(a) **P** is active

Processes

User Stack

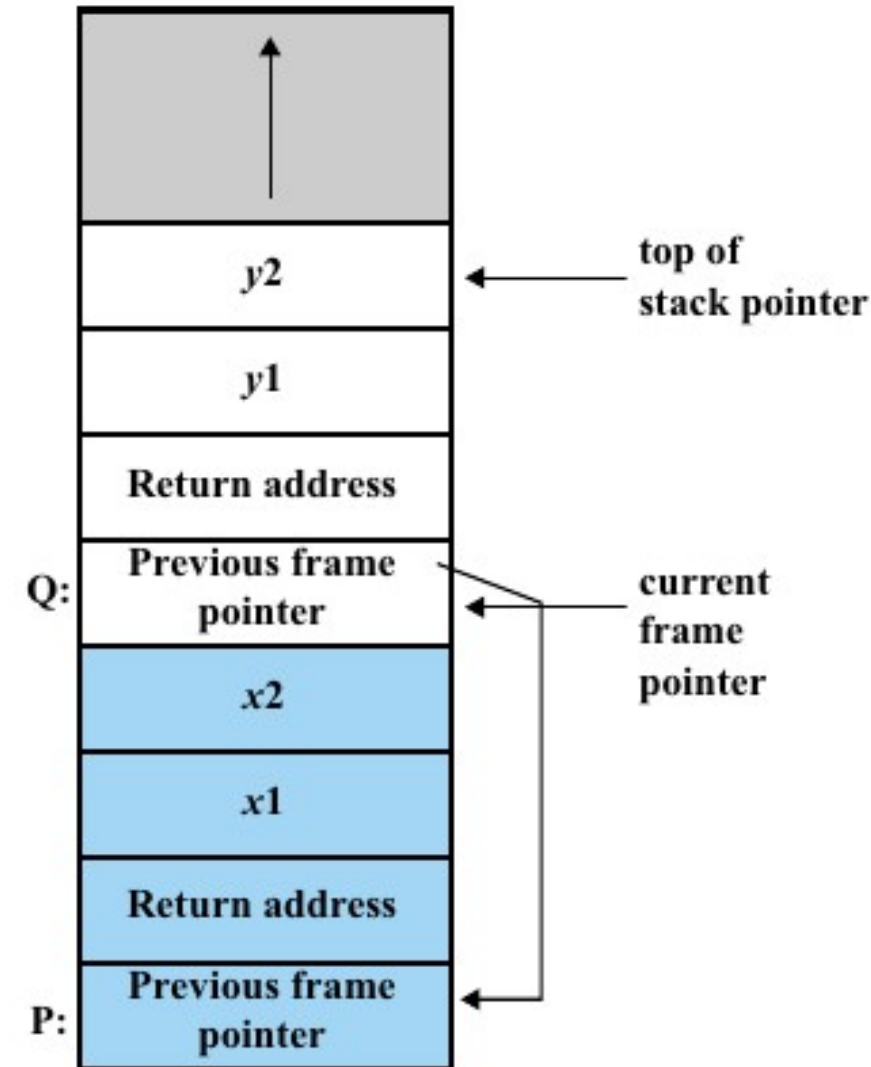


(b) P has called Q

Processes

User Stack

- Supports recursive procedure calls
- Stores of:
 - return address
 - Local variables
 - parameters
 - frame pointer
- Compact usage from memory
- pointer(s) to stack in registers of the processor.



(b) P has called Q

Processes

Process Control Block

Information need by the operating system for process management

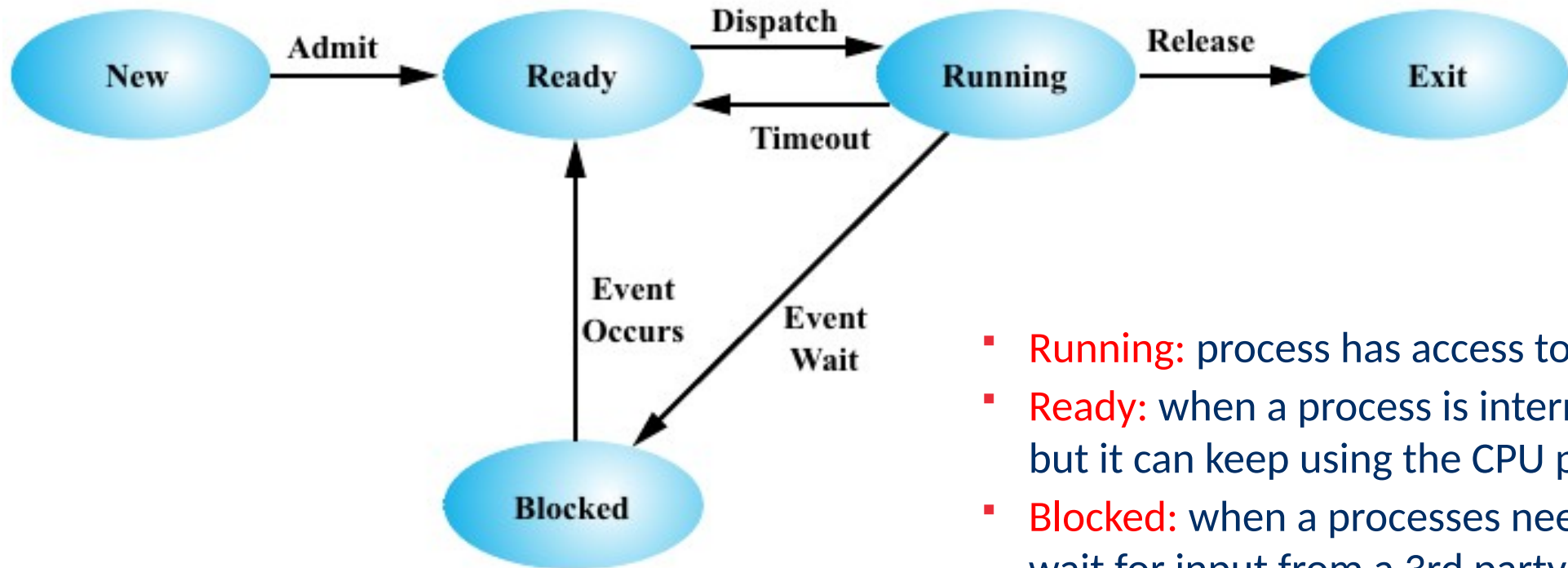
- **Process Identification**
 - Process-id, user-id, parent-id
- **Processor State Information**
 - Related to the processor registers
- **Process Control Information**
 - State, privileges, memory info, etc.

Stored in the process tables of the operating system

Processes

Process States

- Three basic states: *Running*, *Ready* and *Blocked*



- **Running**: process has access to the CPU.
- **Ready**: when a process is interrupted, but it can keep using the CPU properly.
- **Blocked**: when a processes needs to wait for input from a 3rd party.

Processes

Process States: Suspend/Swapped

- Usually we want to have as many active processes as possible to take advantage of the CPU
- When to suspend?

Processes

Process States: Suspend/Swapped

- Usually we want to have as many active processes as possible to take advantage of the CPU
- **When to suspend?**
 - Push a periodic process (e.g. weather checker)
 - A *blocked* process (waiting for feedback) → *blocked-suspend*
 - A *ready* process when another process requires a large block of memory. → *ready-suspend*

Processes

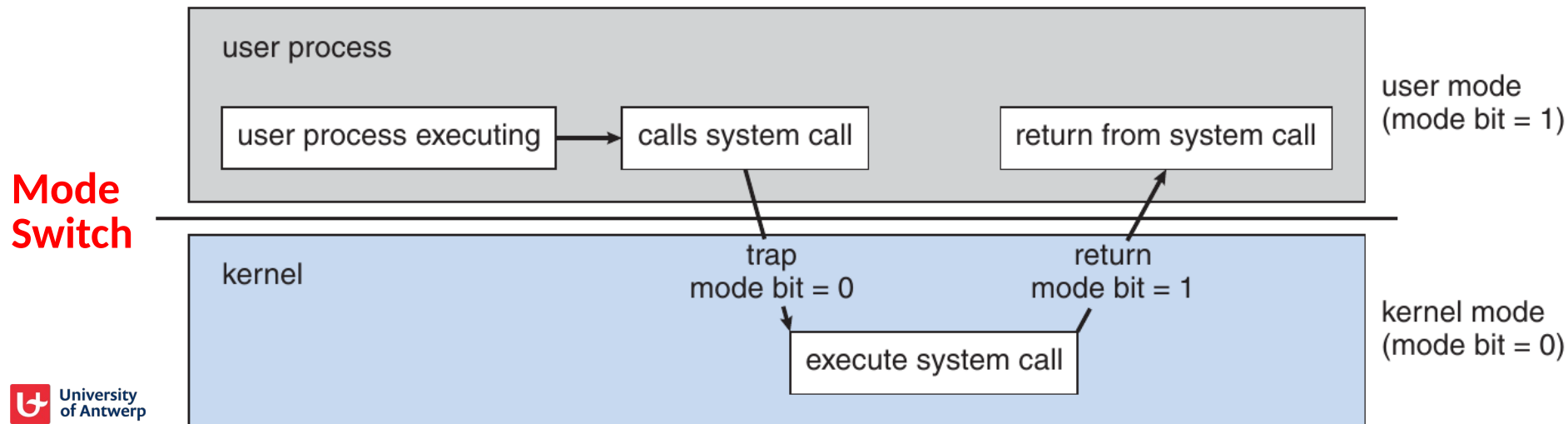
Privilege Levels

- Processors have at least 2 privilege levels:
 - **User mode:** only part of the instructions may be executed.
(some memory might not be addressable)
 - **Kernel mode:** no restriction

Processes

Privilege Levels

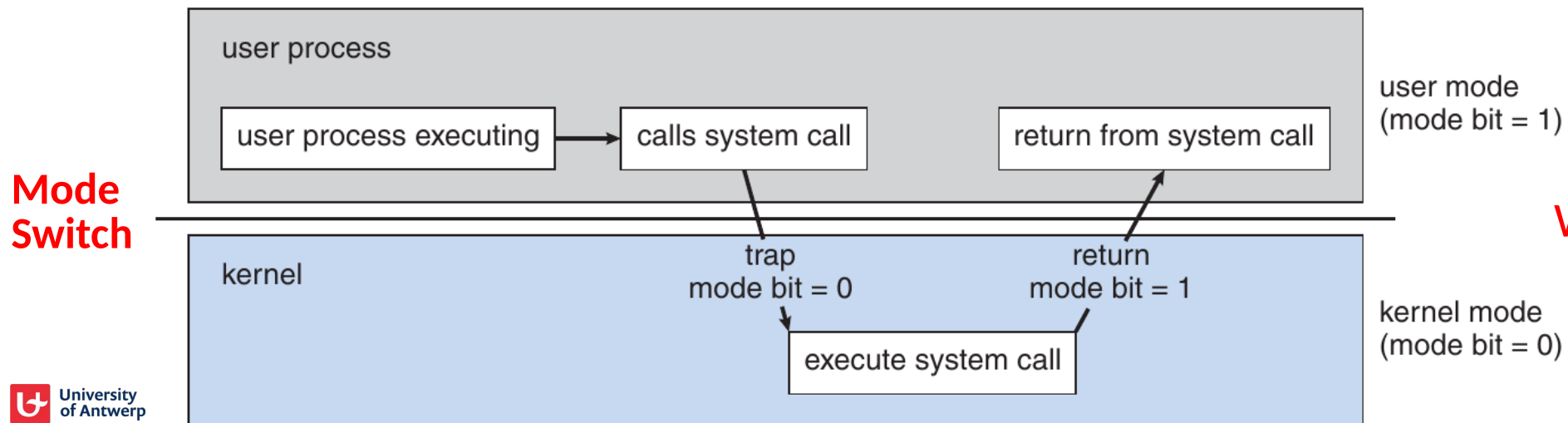
- Processors have at least 2 privilege levels:
 - **User mode:** only part of the instructions may be executed.
(some memory might not be addressable)
 - **Kernel mode:** no restriction



Processes

Privilege Levels

- Processors have at least 2 privilege levels:
 - **User mode:** only part of the instructions may be executed.
(some memory might not be addressable)
 - **Kernel mode:** no restriction



Why is this necessary?



Processes

Mode VS. Process Switch

- **Process Switch:** interruption while *running* process i (timer, I/O, page fault) and next process on the queue: j
 - State information from the registers → “Preempted” state.
 - Update process table for i and j .
- **Mode Switch:** after interrupt handler comes same process again.
 - Just swap state information and execute handler
 - Further tables remain identical, e.g. process state remains *running*

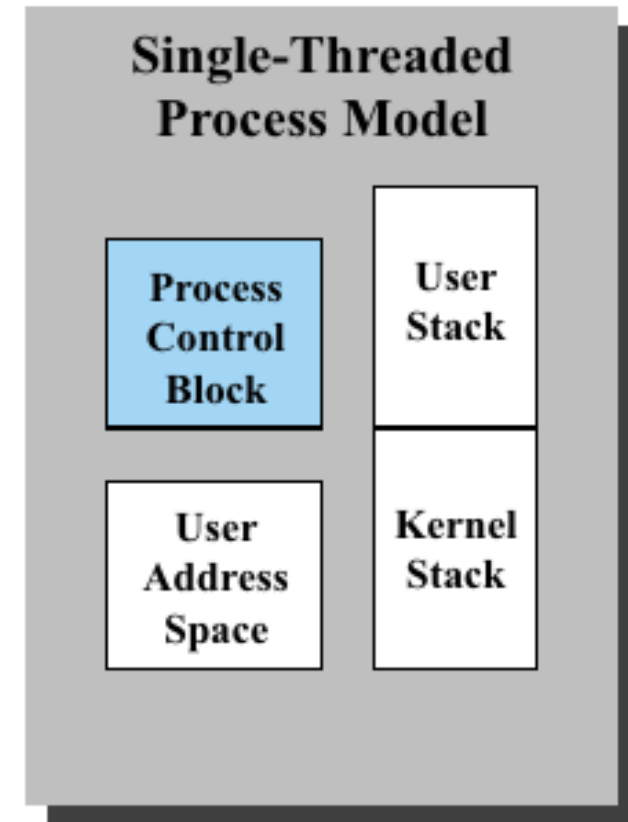
Threads

[further breaking down programs]

Threads

Motivation

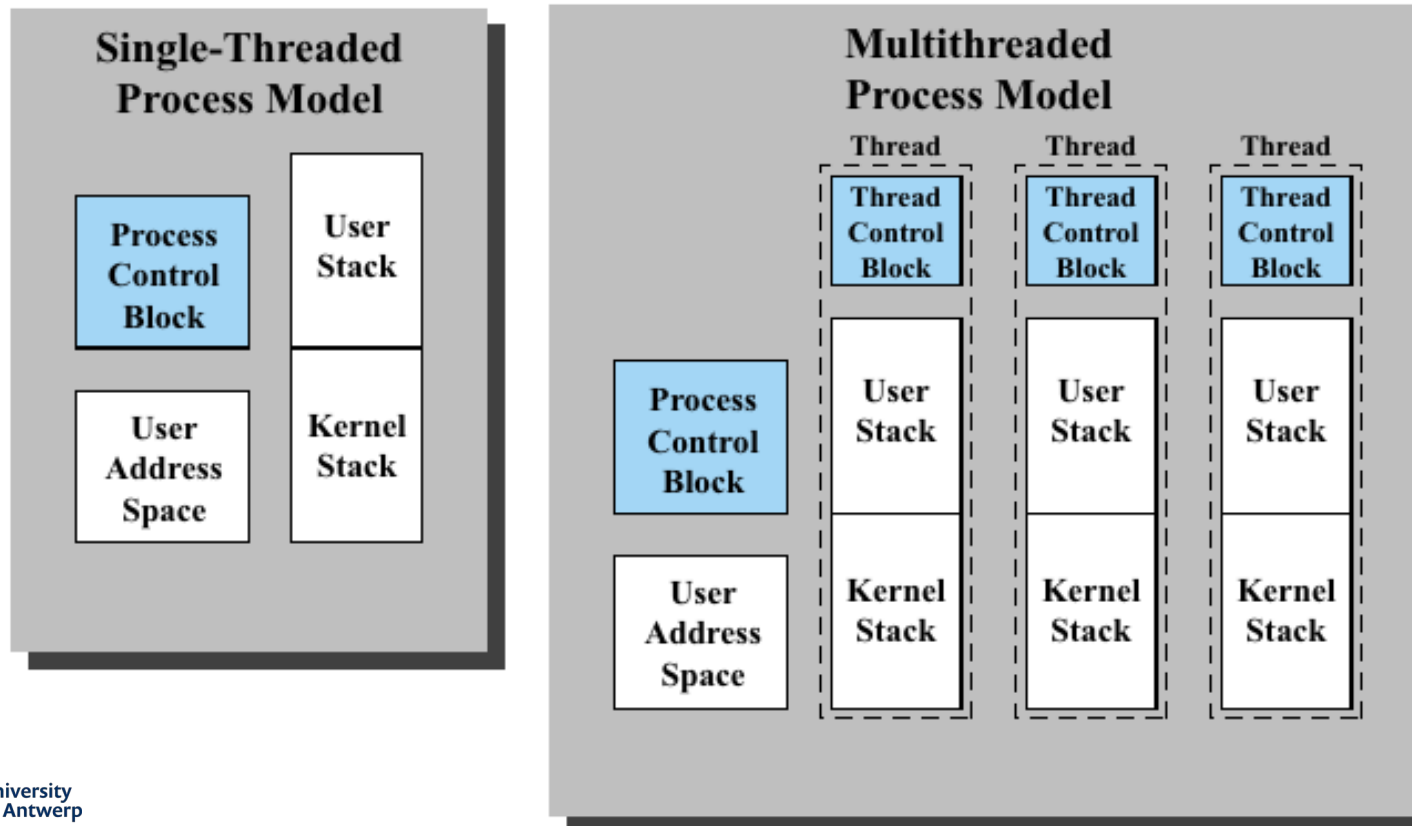
- Each process has several associated resources and a set of instructions that are executed internally.
 - A program could be seen as multiple processes (each with its own process image)
 - Each process with its own control block
 - Joint code and data could be shared
 - > switch to kernel mode to write into shared memory
- Short lifetime will make the added overhead inefficient.



Threads

What

- Partition a program more efficiently



- Share resources (code/data and control block)
- Each process follow a different path of execution (have their own user/kernel stack)

Processes vs. Threads

Multiple Processes

- Slower in creation, switching (10x) (-)
- Requires process communication or sharing, which involves the kernel (-)
- Concurrency guaranteed by OS (+)
- Different process states (+)
- Independently swapped (+)

Processes vs. Threads

Multiple Processes

- Slower in creation, switching (10x) (-)
- Requires process communication or sharing, which involves the kernel (-)
- Concurrency guaranteed by OS (+)
- Different process states (+)
- Independently swapped (+)

Multiple Threads

- Fast in creation, switching (+)
- Immediate information exchange (+)
- Synchronization and mutual exclusion not guaranteed (-)
- One process state (for user-level threads) (-)
- Jointly swapped (-)

Threads

Two Implementations

- **User-level threads (UTL):** thread management is performed by the process to which the threads belong.
 - Kernel is not aware the process is threaded
 - Handled via a thread library.
 - When created, the process will be assigned one thread. Using the libraries it will create new threads.
- **Kernel-level threads (KLT):** thread management handled by the kernel.
 - Scheduling at thread level instead of process level.
 - Low efficiency -> a single thread change involves the kernel and mode switch.

User VS. Kernel -level Threads

ULTs +

- Thread changes are fast
- Application-specific scheduling
- Processor independent

ULTs -

- Process in *blocked* state by calling one thread (unless jacking)
- Can only use 1 processor

User VS. Kernel -level Threads

ULTs +

- Thread changes are fast
- Application-specific scheduling
- Processor independent

ULTs -

- Process in *blocked* state by calling one thread (unless jacking)
- Can only use 1 processor

KLTs +

- Use multiple processors simultaneously
- Single thread *blocked* will not *block* others

KLT -

- Thread change/execution is slower (10-30x) → OS scheduling
- Processor-dependent

Key points

[Finally :D]

Processes & Threads

Pay attention to...

- Main characteristics of Processes and Threads
- Process states and privilege levels.
- Difference between Mode Switch and Process Switch + importance.
- Strengths and Weaknesses of using multiple process and multiple threads.

Further Reading

General OS / Computer Organization Introduction

- Silberschatz et al., Operating System Concepts Essentials, 2nd Edition., **Chapter-1** – *recommended*

Essential: Processes and Threads

- OS Course Notes chapter 1.

If time allows:

- Silberschatz et al., Operating System Concepts Essentials, 2nd Edition., chapter 3 & 4

Questions?



Operating Systems

[1500WETOPS]

José Oramas